Question 1

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

ALGORITHM:

```
#include <stdio.h>

int main() {
    int m;
    scanf("%d", &m);
    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }

    int low = 0, high = m - 1, first_zero = m;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == 0) {
            first_zero = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    printf("%d\n", m - first_zero);
    return 0;
}
```

INPUT:

2
1410
2007

OUTPUT:

0

QUESTION 2:

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]
Output: 3
Example 2:

Input: nums = [2,2,1,1,1,2,2]
Output: 2

Constraints:

n == nums.length
1 <= n <= 5 * 104
-231 <= nums[i] <= 231 - 1

For example:

Input Result
3
3 2 3
3
7
2 2 1 1 1 2 2
2
Answer:(penalty regime: 0 %)

ALGORITHM:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }
```

```
    int candidate = 0, count = 0;
    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = nums[i];
        }
        count += (nums[i] == candidate) ? 1 : -1;
    }

    printf("%d\n", candidate);
    return 0;
}
```

INPUT:

```
4
45
47
85
62
```

OUTPUT:

```
85
```

QUESTION 3:

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer:(penalty regime: 0 %)

ALGORITHM:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
```

```
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int x;
    scanf("%d", &x);

    int low = 0, high = n - 1, floor = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] <= x) {
            floor = arr[mid];
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    printf("%d\n", floor);
    return 0;
}
```

INPUT:

3
14
10
07
20

OUTPUT:

7

QUESTION 4:

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

ALGORITHM:

```
#include <stdio.h>

int search(int arr[], int low, int high, int target) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] > target) return search(arr, low, mid - 1, target);
    return search(arr, mid + 1, high, target);
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    int x;
    scanf("%d", &x);

    for (int i = 0; i < n; i++) {
        int idx = search(arr, i + 1, n - 1, x - arr[i]);
        if (idx != -1) {
            printf("%d\n%d\n", arr[i], arr[idx]);
            return 0;
        }
    }
    printf("No\n");
    return 0;
}
```

ALGORITHM:

```
#include <stdio.h>

int search(int arr[], int low, int high, int target) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] > target) return search(arr, low, mid - 1, target);
    return search(arr, mid + 1, high, target);
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
    int x;
    scanf("%d", &x);

    for (int i = 0; i < n; i++) {
        int idx = search(arr, i + 1, n - 1, x - arr[i]);
        if (idx != -1) {
            printf("%d\n%d\n", arr[i], arr[idx]);
            return 0;
        }
    }
    printf("No\n");
    return 0;
}
```

INPUT:

```
4
12
45
78
65
214
```

OUTPUT:

```
NO
```

QUESTION 5:

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5	

67 34 12 98 78
12 34 67 78 98

ALGORITHM:

```
#include <stdio.h>

void quicksort(int a[], int first, int last) {
    int i, j, pivot, temp;
    if (first < last) {
        pivot = first;
        i = first;
        j = last;
        while (i < j) {
            while (a[i] <= a[pivot] && i < last) i++;
            while (a[j] > a[pivot]) j--;
            if (i < j) {
                temp = a[i]; a[i] = a[j]; a[j] = temp;
            }
        }
        temp = a[pivot]; a[pivot] = a[j]; a[j] = temp;
        quicksort(a, first, j - 1);
        quicksort(a, j + 1, last);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    quicksort(a, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d%s", a[i], (i == n - 1) ? "" : " ");
    }
    printf("\n");
    return 0;
}
```

INPUT:
4
74
65
24
36
OUTPUT:
24 36 65 74